Appunti di Cloud Networking

07 - Virtualization and Containerization

Obiettivo degli appunti

Capire la differenza tra virtualizzazione e containerizzazione, il ruolo degli hypervisor, il funzionamento dei container engine, dei pod, degli host, dei cluster e degli strumenti di gestione/orchestrazione.

Fonte: 07 - CN - Virtualization and Containerization.pdf, slide 1-30.

Indice degli argomenti

- 1. Operating Systems e runtime
- 2. Virtualization: idea generale
- 3. Hypervisor e controllo delle risorse
- 4. Virtualization Type 1 e Type 2
- 5. Containerization: concetto e vantaggi
- 6. Container engine, immagini e piani di controllo
- 7. Container su server fisici e virtuali
- 8. Pod: definizione, IP, storage e deployment
- 9. Host, host cluster e host network
- 10. Container instances, replicas e container clusters
- 11. Container package management
- 12. Container orchestration
- 13. Package manager vs orchestrator
- 14. Glossario essenziale
- 15. Domande d'esame con risposta subito sotto

Come studiare questo capitolo: Studia prima la catena logica: sistema operativo -> virtualizzazione -> hypervisor -> virtual server -> container engine -> container -> pod -> host/cluster -> orchestrator. Le domande d'esame di solito chiedono differenze e responsabilita dei componenti, non solo definizioni isolate.

1. Operating Systems e runtime

Il punto di partenza del capitolo e il sistema operativo. Un operating system e il software installato su un computer che mette a disposizione programmi, strumenti, librerie e risorse per gestire il computer e per supportare le applicazioni installate.

Concetto	Spiegazione
Operating system	Software di base che gestisce il computer e fornisce servizi alle applicazioni.
Runtime	Insieme dei programmi dell'OS che supportano l'esecuzione e il funzionamento attivo delle applicazioni.
Runtime applicativo	Alcune applicazioni introducono anche software runtime proprio, che funziona sopra il runtime del sistema operativo.

Idea da ricordare: Senza un runtime adeguato, l'applicazione non ha l'ambiente necessario per essere eseguita. Virtualizzazione e containerizzazione sono due modi diversi di fornire ambienti di esecuzione.

2. Virtualization: idea generale

La virtualizzazione e la tecnologia che permette a una risorsa IT fisica di fornire piu immagini virtuali di se stessa. In pratica, le capacita di elaborazione di un singolo server fisico possono essere condivise da piu soluzioni indipendenti.

- **Risorsa piu virtualizzata:** il server fisico.
- **Obiettivo:** astrarre l'hardware fisico e creare piu server virtuali.
- **Risultato:** ogni server virtuale puo avere un proprio ambiente operativo dedicato, chiamato guest operating system.
- **Vantaggio:** piu soluzioni possono usare lo stesso hardware fisico senza dover possedere un server dedicato ciascuna.

Schema logico: Server fisico -> hypervisor -> server virtuali -> guest operating system -> applicazioni.

3. Hypervisor e controllo delle risorse

L'hypervisor e il componente responsabile della creazione e dell'esecuzione di piu server virtuali a partire da un server fisico. E il punto centrale della virtualizzazione.

Concetto	Spiegazione
Amministratore del server fisico	Mantiene il controllo amministrativo sull'hardware fisico e, se presente, sul sistema operativo del server fisico.
Amministratori dei server virtuali	Gestiscono i rispettivi ambienti virtuali, ma non hanno accesso diretto al server fisico sottostante.
Hypervisor	Crea e gestisce le macchine virtuali, fornendo a ciascuna un ambiente operativo virtualizzato.

Perche e importante: L'isolamento amministrativo e fondamentale: chi gestisce una VM non deve necessariamente poter modificare l'host fisico. Questo aumenta sicurezza, separazione e controllo.

4. Virtualization Type 1 e Type 2

Le slide distinguono due ambienti di virtualizzazione: Type 1 e Type 2. La differenza principale riguarda la posizione dell'hypervisor rispetto al sistema operativo del server fisico.

Tipo	Struttura	Spiegazione	Conseguenza pratica
Type 1	Server fisico -> hypervisor -> server virtuali -> OS guest	Il server fisico non ha un sistema operativo tradizionale installato sopra l'hardware. L'hypervisor e installato direttamente sul server fisico.	Architettura piu diretta: l'hypervisor controlla direttamente l'hardware e crea i server virtuali.
Type 2	Server fisico -> OS host -> hypervisor -> server virtuali -> OS guest	Il server fisico ha un sistema operativo installato e sopra di esso puo essere installato l'hypervisor.	Il server fisico resta accessibile tramite il proprio OS; la virtualizzazione passa attraverso un livello in piu.

Errore da evitare: Non dire che nel Type 1 manca sempre qualsiasi sistema operativo: manca l'OS host tradizionale sopra cui gira l'hypervisor. Le VM create dall'hypervisor hanno comunque i propri guest operating systems.

5. Containerization: concetto e vantaggi

La containerizzazione nasce storicamente da tecniche Unix degli anni 1970 per separare meglio il codice applicativo. L'idea e fornire un ambiente isolato in cui applicazioni e servizi possono funzionare senza interferire con altri processi.

Un container e un ambiente di hosting virtualizzato ottimizzato per fornire solo le risorse necessarie ai programmi che ospita. A differenza di una macchina virtuale, non contiene una copia completa del sistema operativo.

Aspetto	Virtual server / VM	Container
Sistema operativo	Fornisce una versione virtuale dell'intero sistema operativo.	Fornisce solo il sottoinsieme delle risorse dell'OS richiesto dal software ospitato.
Peso	Piu pesante, perche include un ambiente operativo completo.	Piu leggero, perche usa meno componenti.
Efficienza	Buona separazione, ma con maggiore overhead.	Maggiore efficienza e minore consumo di spazio.
Caso tipico	Quando serve un ambiente OS completo e isolato.	Quando serve distribuire rapidamente applicazioni o servizi con dipendenze definite.

Frase da esame: La VM virtualizza un server completo; il container virtualizza un ambiente di esecuzione piu leggero, basato solo sulle risorse OS necessarie all'applicazione.

6. Container engine, immagini e piani di controllo

Il container engine, detto anche containerization engine, e responsabile della creazione dei container a partire da immagini predefinite. Le immagini possono essere viste come template da cui vengono creati container eseguibili.

Concetto	Spiegazione
Container image	Template predefinito che contiene cio che serve per creare un container.
Container engine	Componente che crea e gestisce container a partire dalle immagini.
Management plane	Parte composta da strumenti GUI e CLI usati dall'amministratore per configurare e mantenere l'ambiente.
Control plane	Parte composta dalle funzioni automatiche del motore, eseguite in risposta ai comandi e alle configurazioni.

Schema logico: Immagine container -> container engine -> container in esecuzione. L'amministratore usa il management plane; il motore applica le azioni tramite il control plane.

7. Container su server fisici e virtuali

I container possono essere distribuiti sia direttamente su server fisici sia sopra server virtuali. La differenza sta nella presenza o meno dello strato di virtualizzazione.

Scenario	Struttura	Caratteristica principale
Container su server fisico	Server fisico -> OS -> containerization platform -> container	Non serve un ambiente di virtualizzazione, perche non vengono create VM. Il container usa un sottoinsieme delle risorse dell'OS del server fisico.
Container su virtualizzazione Type 1	Server fisico -> hypervisor -> VM con OS -> containerization platform -> container	Ogni VM ospita una piattaforma di containerizzazione che crea container con sottoinsieme OS.
Container su virtualizzazione Type 2	Server fisico -> OS host -> hypervisor -> VM con OS -> containerization platform -> container	E presente sia l'OS host del server fisico sia l'OS delle VM; lo strato complessivo e piu articolato.

Attenzione prestazioni: Quando un pod o un container viene eseguito sopra un server virtuale, lo strato di virtualizzazione aggiuntivo puo introdurre latenza di elaborazione. Le prestazioni vanno misurate prima di scegliere il deployment migliore.

8. Pods: definizione, IP, storage e deployment

Un pod, chiamato anche logical pod container, e un tipo speciale di system container che puo ospitare un singolo container oppure un gruppo di container. I container dentro lo stesso pod condividono risorse di rete, storage e configurazione di esecuzione.

- **Condivisione rete:** i container di uno stesso pod possono usare lo stesso indirizzo IP.
- **Condivisione storage:** il pod puo fornire uno storage comune, spesso esposto come file system chiamato volume.
- **Relazione logica:** i container raggruppati nel pod sono spesso parte della stessa soluzione distribuita o dello stesso namespace.
- **Comunicazione interna:** i container possono comunicare tramite meccanismi standard di interprocess communication, come memoria condivisa.
- **Limite importante:** un singolo pod non puo estendersi su piu host.

8.1 Sequenza di deployment di un pod

1. L'amministratore crea prima un pod vuoto.
2. Il pod riceve una configurazione, ad esempio indirizzo IP del pod e risorse condivise.
3. L'amministratore istruisce il container engine ad aggiungere container al pod.
4. I container vengono eseguiti nel pod e possono condividere rete, storage e configurazione.

Perche i pod sono utili: Permettono di mantenere insieme componenti software correlati, soprattutto quando devono condividere IP, storage o configurazioni. Sono anche collegati a funzionalita di container chains, orchestrazione e scaling.

9. Hosts, host cluster e host network

Un host e l'ambiente in cui un container viene distribuito. Puo essere chiamato server o node. L'host fornisce il sistema operativo da cui il container astrae le risorse necessarie.

Concetto	Spiegazione
Host	Server o nodo su cui vengono eseguiti container e pod.
Nodo	Nome spesso usato per indicare un host dentro un ambiente cluster.
Più container per host	Un singolo host puo eseguire piu container.
Un pod non attraversa piu host	Un singolo pod deve risiedere su un solo host.
Container senza pod	Possibile quando il container engine non supporta i pod.

9.1 Host clusters

Gli host server possono essere combinati in host cluster. Un cluster crea un pool di risorse di elaborazione disponibili e aumenta la capacita complessiva.

Tipo di cluster	Funzione	Quando serve
Load-balanced cluster	Distribuisce i workload tra piu host, aumentando la capacita mantenendo centralizzata la gestione.	Quando bisogna bilanciare il carico tra piu nodi.
High Availability cluster	Mantiene la disponibilita anche in caso di guasti di piu host. Usa ridondanza e failover automatico.	Quando la continuita del servizio e prioritaria.
Scaling cluster	Supporta scaling verticale e orizzontale.	Quando il sistema deve adattarsi dinamicamente alla domanda.

9.2 Host networks

- **Local host network:** permette la comunicazione tra container correlati nello stesso host.
- **Overlay network:** permette la comunicazione tra container e container engine su host differenti.
- **Container networks:** sia le reti locali dell'host sia le overlay network sono considerate reti di container.

10. Container instances, replicas e container clusters

Dello stesso container possono essere create piu istanze con lo stesso software. Questo permette a piu consumer program di usare contemporaneamente lo stesso servizio applicativo.

Concetto	Spiegazione
Container instance	Istanza concreta di un container.
Replica	Nome comune per indicare un'istanza duplicata dello stesso container.
Container cluster	Pool di istanze container create prima dell'uso effettivo, spesso per esigenze di performance.

I container clusters possono essere creati manualmente o automaticamente. Possono restare caricati in memoria in stato inattivo, pronti a essere invocati. Possono anche essere pianificati per esistere in memoria solo in determinati periodi, per esempio durante picchi previsti di utilizzo.

Motivo principale: Il container cluster serve a ridurre il tempo necessario per fornire istanze containerizzate quando la domanda cresce rapidamente.

11. Container package management

Il container package management riguarda la gestione dei pacchetti software e delle dipendenze nelle applicazioni containerizzate. Non si occupa solo di lanciare container: gestisce immagini, dipendenze e repository.

- **Strumenti CLI:** creano, taggano e inviano immagini container a un repository.
- **Container registry:** repository in cui vengono conservate le immagini container.
- **Container deployment file:** pacchetto che contiene la workflow logic per il deployment.
- **Package repository:** luogo da cui viene recuperato il pacchetto di deployment.
- **Deployment optimizer:** programma che analizza il contenuto del pacchetto e gli host disponibili nel cluster per scegliere la destinazione migliore dei container.

Sequenza da ricordare: Repository -> package/deployment file -> container package manager -> deployment optimizer -> scelta degli host -> distribuzione di pod e container.

12. Container orchestration

La container orchestration e il processo di automazione del deployment, dello scaling e della gestione di applicazioni containerizzate in un ambiente distribuito. Richiede uno strumento chiamato container orchestrator, container orchestration tool o container orchestration platform.

Operazione dell'orchestrator	Cosa fa	Perche e utile
Container deployment	Distribuisce container su piu nodi del cluster.	Permette di usare piu host in modo coordinato.
Load balancing	Distribuisce il traffico tra piu container che eseguono la stessa applicazione.	Aumenta disponibilita e scalabilita.
Scaling	Scala automaticamente up, down, in o out il numero di container in base alla domanda.	Migliora utilizzo risorse e cost efficiency.
Health monitoring	Controlla lo stato dei container e puo riavviare o sostituire quelli guasti.	Riduce downtime e interventi manuali.
Service discovery	Mantiene un service registry per far scoprire e comunicare le applicazioni.	Evita configurazioni manuali rigide.
Storage orchestration	Assicura che i dati siano salvati e recuperati correttamente.	Gestisce persistenza e accesso ai dati.
Network orchestration	Gestisce il networking dei container.	Permette comunicazione corretta tra componenti.
Configuration management	Gestisce configurazioni e puo applicare modifiche ai container in esecuzione.	Riduce errori e facilita aggiornamenti.

13. Package manager vs orchestrator

Le ultime slide distinguono container package manager e container orchestrator. La differenza e importante per l'esame perche i due strumenti agiscono a livelli diversi.

Aspetto	Container Package Manager	Container Orchestrator
Funzione	Gestisce immagini container e relative dipendenze.	Automatizza deployment, scaling e gestione delle applicazioni containerizzate.
Scope	Si concentra sulle immagini e sulle dipendenze.	Gestisce l'intera applicazione containerizzata.
Livello di astrazione	Opera a un livello piu basso.	Fornisce una vista di alto livello dell'intera applicazione.
Toolset	Set limitato di strumenti dedicati a immagini e dipendenze.	Ampio insieme di strumenti e API per gestire container, reti, storage e infrastruttura.

Regola pratica: Il package manager prepara e gestisce cio che serve per costruire e distribuire container; l'orchestrator governa il comportamento dell'applicazione containerizzata nel tempo.

14. Mappa finale da esame

La catena concettuale del capitolo puo essere riassunta cosi:

Livello	Componente	Ruolo
1	Server fisico	Fornisce hardware e capacita di calcolo.
2	Sistema operativo o hypervisor	Fornisce ambiente host oppure livello di virtualizzazione.
3	Virtual server	Ambiente virtuale con proprio guest operating system.
4	Containerization platform / engine	Crea container a partire da immagini.
5	Container	Ambiente leggero che contiene il sottoinsieme OS necessario all'applicazione.
6	Pod	Raggruppa uno o piu container con rete, storage e configurazione condivisi.
7	Host / node	Ambiente su cui pod e container vengono eseguiti.
8	Cluster	Insieme di host o container per capacita, disponibilita, bilanciamento e scaling.
9	Orchestrator	Automatizza deployment, scaling, load balancing, monitoraggio e configurazione.

15. Glossario essenziale

Termine	Definizione
Operating system	Software di base che gestisce un computer e supporta le applicazioni.
Runtime	Componenti software che permettono l'esecuzione attiva delle applicazioni.
Virtualization	Tecnologia che consente a una risorsa fisica di fornire piu immagini virtuali di se stessa.
Physical server	Server reale su cui vengono installati OS, hypervisor o piattaforme di containerizzazione.
Virtual server	Server virtuale creato tramite hypervisor, con proprio guest OS.
Guest operating system	Sistema operativo eseguito dentro una macchina/server virtuale.
Hypervisor	Componente che crea e gestisce server virtuali da un server fisico.
Container	Ambiente di hosting virtualizzato e leggero, con solo le risorse OS necessarie.
Container image	Template da cui viene creato un container.
Container engine	Motore che crea e gestisce container a partire da immagini.
Management plane	Interfacce GUI/CLI usate per configurare il container engine.
Control plane	Funzioni automatiche del container engine.
Pod	Contenitore logico che ospita uno o piu container con risorse condivise.
Volume	File system o storage condiviso fornito a container nello stesso pod.
Host / node	Server o ambiente in cui sono distribuiti container e pod.
Host cluster	Gruppo di host che fornisce un pool di risorse disponibili.
Overlay network	Rete che collega container o container engine distribuiti su host diversi.
Replica	Istanza duplicata dello stesso container.
Container registry	Repository in cui vengono conservate immagini container.
Container package manager	Strumento per gestire immagini container e dipendenze.
Container orchestrator	Strumento che automatizza deployment, scaling e gestione dei container.

16. Domande d'esame con risposta subito sotto

Domanda 1: Che cos'e un sistema operativo nel contesto del capitolo?

Risposta: E il software installato su un computer che fornisce programmi, strumenti, librerie e risorse per gestire il computer e supportare le applicazioni. I programmi dell'OS che supportano l'esecuzione delle applicazioni formano il runtime.

Domanda 2: Che cosa si intende per virtualizzazione?

Risposta: La virtualizzazione e la tecnologia che permette a una risorsa IT fisica, tipicamente un server fisico, di fornire piu immagini virtuali di se stessa, condividendo le proprie capacita tra piu soluzioni.

Domanda 3: Qual e il ruolo dell'hypervisor?

Risposta: L'hypervisor crea ed esegue piu server virtuali a partire da un server fisico. Permette agli amministratori delle VM di controllare i propri ambienti virtuali senza accedere direttamente all'hardware fisico.

Domanda 4: Qual e la differenza tra virtualizzazione Type 1 e Type 2?

Risposta: Nel Type 1 l'hypervisor e installato direttamente sul server fisico, senza un OS host tradizionale. Nel Type 2 il server fisico ha un sistema operativo installato e l'hypervisor gira sopra tale sistema operativo.

Domanda 5: Che cos'e un guest operating system?

Risposta: E il sistema operativo eseguito dentro un server virtuale. Ogni virtual server puo avere una propria copia dedicata dell'ambiente operativo.

Domanda 6: Che cos'e un container?

Risposta: Un container e un ambiente di hosting virtualizzato ottimizzato, che fornisce solo il sottoinsieme delle risorse del sistema operativo necessario ai programmi che ospita.

Domanda 7: Perche un container e piu leggero di una macchina virtuale?

Risposta: Perche una VM fornisce una versione virtuale dell'intero sistema operativo, mentre un container usa solo le risorse OS effettivamente richieste dal software ospitato.

Domanda 8: Che cos'e un container engine?

Risposta: E il componente responsabile della creazione e gestione dei container a partire da immagini predefinite. E organizzato in management plane e control plane.

Domanda 9: Che differenza c'e tra management plane e control plane nel container engine?

Risposta: Il management plane comprende GUI e strumenti da linea di comando usati per configurare e mantenere l'ambiente. Il control plane comprende le funzioni automatiche del motore che rispondono a configurazioni e comandi.

Domanda 10: I container richiedono sempre una macchina virtuale?

Risposta: No. Possono essere distribuiti direttamente su server fisici con sistema operativo e containerization platform. Possono anche essere distribuiti dentro server virtuali.

Domanda 11: Che cos'e un pod?

Risposta: Un pod e un contenitore logico che puo ospitare uno o piu container che condividono risorse di storage, rete e configurazione di esecuzione.

Domanda 12: Perche si usano i pod?

Risposta: Si usano per raggruppare container correlati, ad esempio componenti della stessa soluzione distribuita, soprattutto quando devono condividere IP, storage o configurazioni.

Domanda 13: Un pod puo estendersi su piu host?

Risposta: No. Un singolo pod non puo span more than one host, cioe non puo estendersi su piu host.

Domanda 14: Che cos'e un host?

Risposta: Un host e l'ambiente, server o nodo, in cui viene distribuito un container. Fornisce il sistema operativo da cui il container astrae le risorse necessarie.

Domanda 15: Quali sono i principali tipi di host cluster?

Risposta: Load-balanced cluster, High Availability cluster e Scaling cluster. Il primo distribuisce il carico, il secondo mantiene la disponibilita tramite ridondanza e failover, il terzo supporta scaling verticale e orizzontale.

Domanda 16: Che differenza c'e tra local host network e overlay network?

Risposta: La local host network collega container nello stesso host. L'overlay network collega container e container engine su host differenti.

Domanda 17: Che cosa sono le replicas?

Risposta: Sono istanze multiple dello stesso container con lo stesso software, create per permettere uso concorrente e maggiore disponibilita del servizio.

Domanda 18: Che cos'e un container cluster?

Risposta: E un pool di container instances istanziate in anticipo rispetto all'uso effettivo, spesso per supportare requisiti di alta performance e provisioning rapido.

Domanda 19: Che cosa fa un container package manager?

Risposta: Gestisce immagini container, dipendenze, repository e workflow di deployment. Include strumenti per creare, taggare e inviare immagini a un container registry.

Domanda 20: Che cosa fa un container orchestrator?

Risposta: Automatizza deployment, scaling e gestione di applicazioni containerizzate in ambienti distribuiti. Gestisce anche load balancing, health monitoring, service discovery, storage, networking e configurazione.

Domanda 21: Qual e la differenza principale tra package manager e orchestrator?

Risposta: Il package manager si concentra su immagini e dipendenze; l'orchestrator gestisce l'intera applicazione containerizzata durante il suo ciclo operativo.

17. Riassunto finale ultracompatto

Concetto	Da ricordare in una frase
Virtualizzazione	Crea piu server virtuali da un server fisico tramite hypervisor.
Hypervisor	Gestisce l'astrazione dell'hardware e la creazione delle VM.
Type 1	Hypervisor direttamente sul server fisico.
Type 2	Hypervisor sopra un sistema operativo host.
Containerizzazione	Crea ambienti leggeri che usano solo il sottoinsieme OS necessario.
Container engine	Crea container da immagini predefinite.
Pod	Raggruppa container con rete, storage e configurazione condivisi.
Host	Nodo/server dove vengono eseguiti container e pod.
Cluster	Pool di host o container per capacita, alta disponibilita e scaling.
Package manager	Gestisce immagini e dipendenze.
Orchestrator	Gestisce automaticamente deployment, scaling e vita operativa dei container.

Priorita per l'esame: Saper confrontare: VM vs container, hypervisor Type 1 vs Type 2, package manager vs orchestrator, host vs pod, replica vs container cluster.